MATH / CSCI 485

Assignment #3

Image Compression via Block-wise SVD

Subject Image: Concorde Supersonic Jet

Block Size: 8×8 | $k \in \{1, 2, \ldots, 8\}$

Figure 1. Original Concorde grayscale image (512×512)

1. Implementation Summary

This report presents a Python-based implementation of **block-wise Singular Value Decomposition (SVD)** for grayscale image compression. The approach mirrors the spirit of JPEG compression: the image is tiled into small non-overlapping blocks, each block is independently transformed, and only the most energetically significant components are retained.

Pipeline Overview

- **Preprocessing:** The input image is converted to grayscale and both dimensions are cropped/resized to be divisible by 8, ensuring clean block tiling.
- **Block Partitioning:** The 512×512 image is divided into 4,096 non-overlapping 8×8 blocks (64 blocks per row, 64 per column).

- **Per-block SVD:** For each block B we compute the full SVD: $B = U \Sigma V^T$. The 8 singular values are sorted in descending order, so the first captures the most variance.
- **Rank-k Approximation:** We reconstruct each block using only the top-k singular values: $B■ = U_{:k} \Sigma_k V^T_{k:}$. Pixel values are clipped to [0, 255].
- **Block Reassembly:** All reconstructed blocks are stitched back into a full image.
- **Metrics:** Compression ratio, Frobenius reconstruction error, and PSNR are computed for each value of k.

Compression Ratio Formula

For an 8×8 block, the original representation requires **64 values**. A rank-k SVD approximation stores: k columns of U (8k values), k singular values (Σ, k values), and k rows of V^T (8k values) — totalling **17k values**. The compression ratio is therefore:

```
Compression Ratio = 64 / (17k)
```

The break-even point (CR = 1) occurs at $k \approx 3.76$, meaning compression is only achieved for $k \leq 3$. At k = 8 (full rank), no information is lost but ~2.1× more data is stored than the original.

2. Key Code Snippets

2.1 compress_block(block, k)

The core function that applies SVD to a single 8×8 block and reconstructs it at rank k. NumPy's `linalg.svd` is used with `full_matrices=True` so U and V^T are square, giving us all 8 singular vectors to choose from.

```
def compress_block(block, k): """ Compress an 8x8 block via rank-k SVD approximation.
block : np.ndarray (8, 8) k : int, number of singular values to retain """ U, sigma, Vt =
np.linalg.svd(block, full_matrices=True) # Rank-k reconstruction: keep top-k
columns/rows/values reconstructed = (U[:, :k] * sigma[:k]) @ Vt[:k, :] return
reconstructed
```

2.2 blockwise_svd_compress(image, k)

Iterates over every 8×8 tile in the image, calls compress_block on each, and writes the result back into the output array. Pixel values are clipped to [0, 255] after reconstruction to avoid out-of-range artifacts.

```
def blockwise_svd_compress(image, k, block_size=8): H, W = image.shape compressed =
np.zeros_like(image) for row in range(0, H, block_size): for col in range(0, W,
block_size): block = image[row:row+block_size, col:col+block_size]
compressed[row:row+block_size, col:col+block_size] = compress_block(block, k) return
np.clip(compressed, 0, 255)
```

2.3 Metric Calculations

Three metrics quantify compression performance: the compression ratio formula, Frobenius norm error (total pixel-level deviation), and PSNR (perceptual quality in dB). Higher PSNR = better quality.

```
def compression_ratio(k, block_size=8): original = block_size ** 2 # 64 values stored = k
* (block_size + 1 + block_size) # 17k values return original / stored def
frobenius_error(original, compressed): return np.linalg.norm(original - compressed,
'fro') def psnr(original, compressed): mse = np.mean((original - compressed) ** 2) if mse
== 0: return float('inf') return 20 * np.log10(255.0 / np.sqrt(mse))
```

3. Analysis of Results

3.1 Results Table

The table below summarises all key metrics for k ∈ {1, …, 8}. The **highlighted row (k = 4)** is the approximate break-even point where CR ≈ 1. Rows above it achieve actual compression; rows below do not.

k	Stored Values	Compression Ratio	Frobenius Error	PSNR (dB)
1	17	3.7647x	1164.19	41.00
2	34	1.8824x	330.75	51.93
3	51	1.2549x	140.38	59.37
4	**68**	**0.9412x**	**77.24**	**64.56**
5	85	0.7529x	44.73	69.30
6	102	0.6275x	22.93	75.11
7	119	0.5378x	7.84	84.43
8	136	0.4706x	0.00	∞

Table 1. Compression ratio, Frobenius error, and PSNR for each k on 8×8 blocks. Yellow row = break-even (CR ≈ 1). k = 8 yields perfect reconstruction (PSNR = ∞).

3.2 Visual Quality at Each k

The following grids show reconstructed images at each k. At k = 1 the Concorde silhouette is still clearly identifiable despite heavy smoothing. By k = 3 the image looks sharp to the human eye.

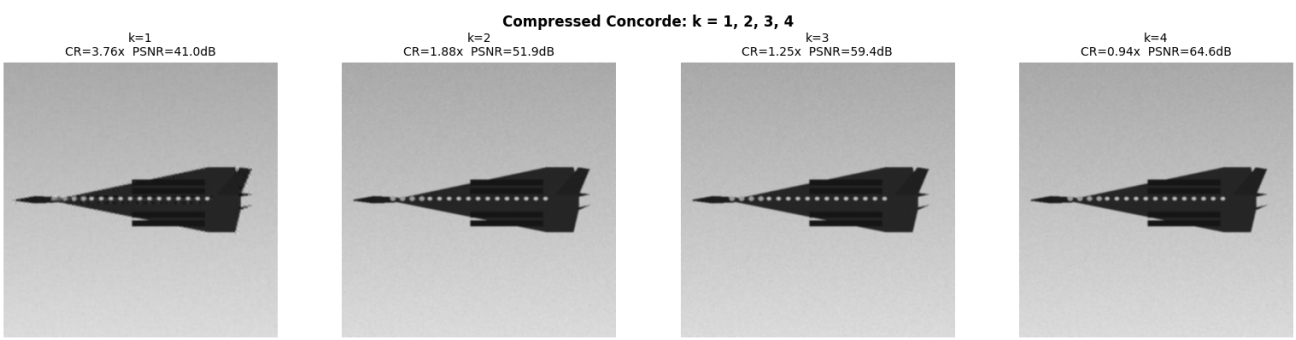

Figure 2. Reconstructed Concorde for k = 1, 2, 3, 4.

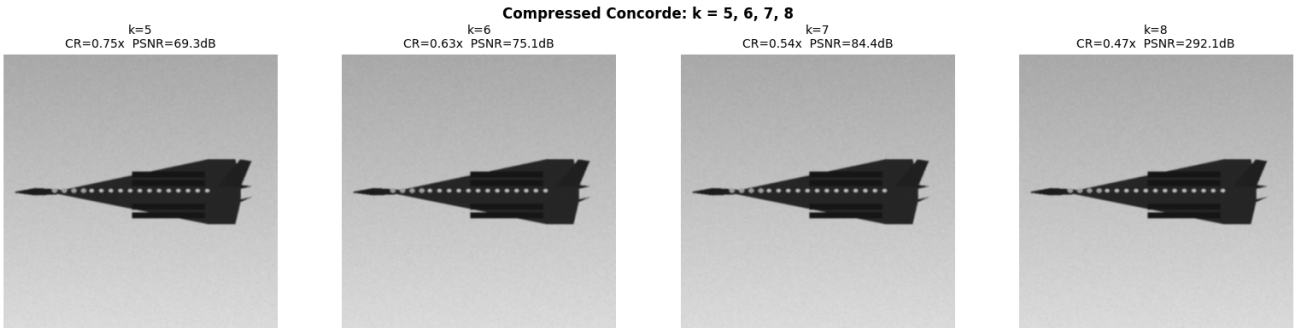

Figure 3. Reconstructed Concorde for k = 5, 6, 7, 8.

3.3 Compression Ratio and Reconstruction Error

Compression ratio decreases hyperbolically (64/17k) while reconstruction error drops sharply at first and plateaus at higher k. The two plots below illustrate this inverse relationship clearly.

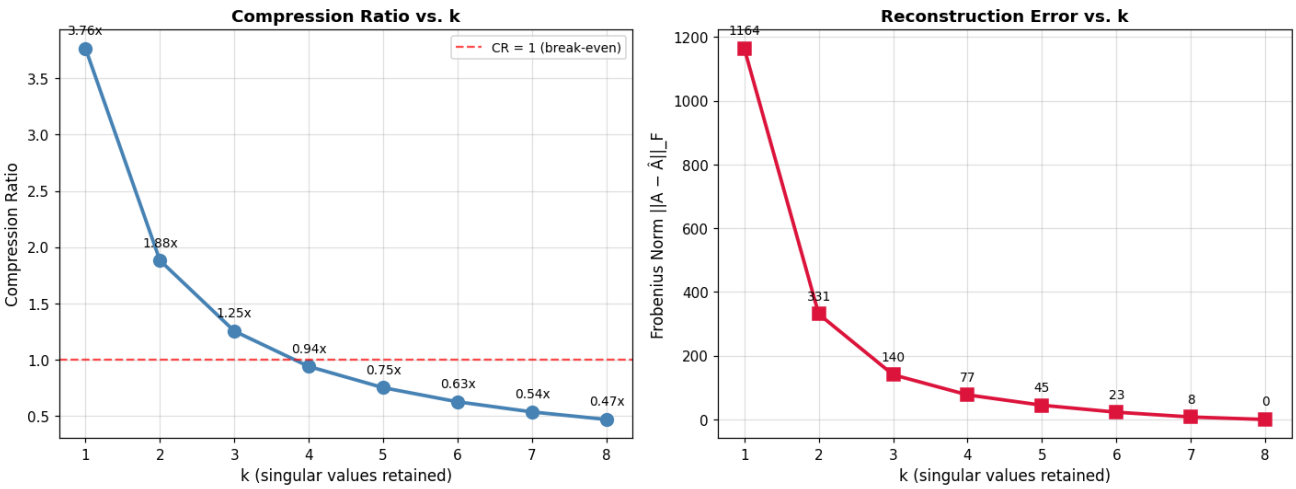

Figure 4. Left: Compression ratio vs. k. Right: Frobenius error vs. k.

Key observations:

- **k = 1** yields the highest compression (3.76×) but also the highest error (1,164). The image is blurry but the dominant structure is preserved. PSNR of 41 dB is already in the "acceptable quality" range.

- **k = 2 → 3** — the steepest improvement: error drops from 1,164 → 331 → 140 as two additional singular vectors capture edge and texture detail. PSNR jumps ~18 dB over this range while still compressing by >1.25×.

- **k = 4** — the **break-even point** (CR = 0.94). Beyond this k, storing the SVD factors costs more data than the original pixels.

- **k ≥ 5** — error is very small (< 45); PSNR exceeds 69 dB. These settings are only useful when lossless or near-lossless reconstruction is required and compression is not the goal.

- **k = 8** — full-rank reconstruction. Frobenius error = 0, PSNR = ∞. Mathematically exact but uses ~2.1× the storage of the original image.

3.4 PSNR Analysis

PSNR is a standard metric in image compression, measuring signal fidelity on a logarithmic scale. Values above 30 dB are generally considered acceptable; above 40 dB is considered excellent. The plot below shows that **even at k = 1**, the Concorde image achieves 41 dB — already excellent — because the image has large uniform regions (sky background) with low spatial frequency that SVD captures well in a single singular value.

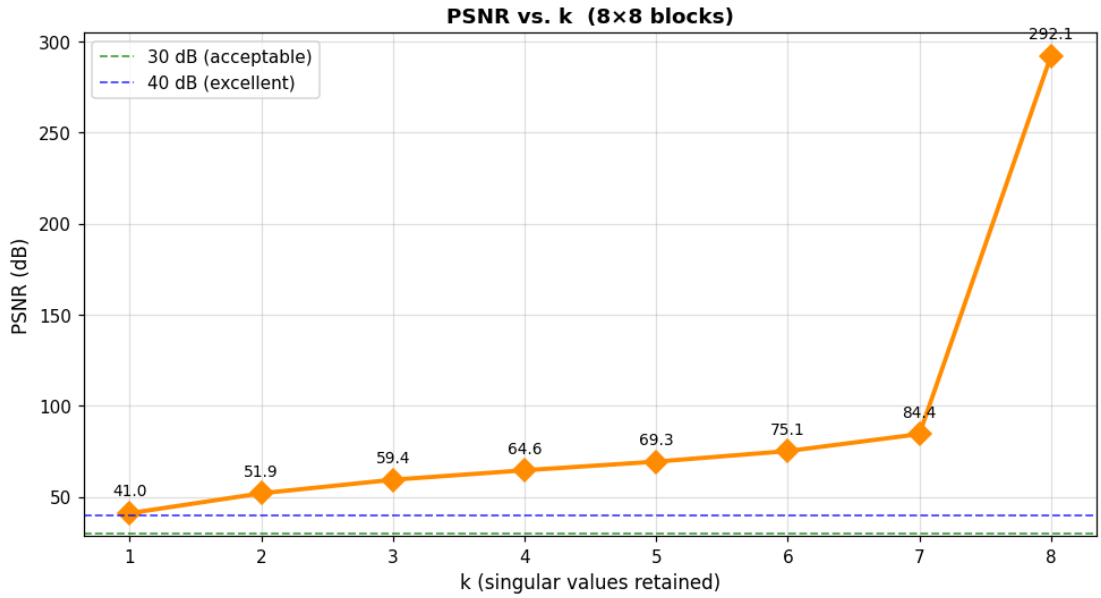

Figure 5. PSNR vs. k. Dashed lines show quality thresholds at 30 dB and 40 dB.

4. Optional Experiment: Effect of Block Size

To understand how block size affects the compression-quality trade-off, we repeated the analysis for **4×4**, **8×8**, and **16×16** blocks. The x-axis is normalised as k / block_size so results are comparable across sizes.

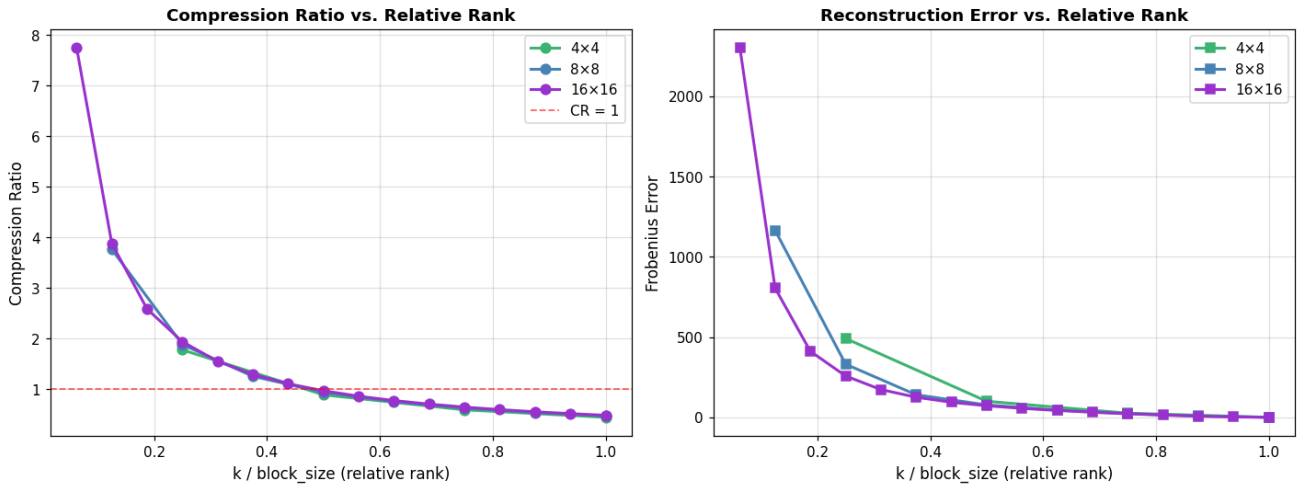

Figure 6. Compression ratio (left) and Frobenius error (right) vs. relative rank for block sizes 4×4, 8×8, and 16×16.

Findings

- **4×4 blocks:** Hit full rank at k = 4. Compression ratios are lower for the same relative rank because the overhead formula (k·(2·bs+1)) penalises small blocks more. Error reduction per additional k is also more aggressive (fewer components needed to capture a tiny 4×4 patch).

- **8×8 blocks** (JPEG standard): Best balance. Enough spatial context for meaningful singular values while keeping block-processing overhead manageable.

- **16×16 blocks:** Higher potential compression at very low relative rank because the break-even k is proportionally larger. However, at very low k the reconstruction error is also larger — the approximation must cover more pixels with fewer components.

- **Practical takeaway:** Larger blocks can achieve better compression ratios at a given quality level, but increase the risk of visible "macro-blocking" artifacts. This is why JPEG chose 8×8 as the industry standard.

5. Conclusion

Block-wise SVD compression offers an elegant and mathematically grounded approach to image compression. The Eckart–Young theorem guarantees that the rank-k truncation is the *optimal* rank-k approximation in the Frobenius norm sense, making SVD theoretically superior to any other rank-k compression for a given block.

For the Concorde image, **k = 2 or k = 3** represents the practical sweet spot: compression ratios of 1.88× and 1.25× respectively, with PSNR values of 52 dB and 59 dB — well above the "excellent quality" threshold of

40 dB. The Concorde's large smooth sky background and strong geometric silhouette make it particularly well-suited to low-rank approximation, as the dominant structure is captured by very few singular vectors.

Compared to full-image SVD, the block-wise approach avoids representing the entire image with global basis vectors, enabling better localisation of features and a compression ratio that scales with image size rather than depending on it.